Predicting Buy and Sell Signals and Expected Returns in the Bitcoin Market

Machine Learning with Business Data, Project Id 3

Krrish Singla (PES1UG24AM141), Shesanth K (PES1UG24AM261), Shamika Mahesh (PES1UG24AM259)

Summary

We built models on hourly Bitstamp Bitcoin data (Dec 2011 to Dec 2012) to classify the market as Buy or Sell and to estimate the expected next-hour return. Five frameworks (XGBoost, Keras, PyTorch with fastai, PyTorch with TabNet, PyTorch Lightning) were compared on simplicity, transparency, efficacy, RMSE, R-squared and MAE. All five recognise the current Buy/Sell trend well (94 to 97% accuracy against an 84.5% baseline) and give a similar small gain on next-hour return (R-squared 0.21 to 0.23 against 0). No framework wins consistently; XGBoost is the fastest and easiest to explain.

1. Introduction

Bitcoin trades around the clock and moves quickly, so automated signals are useful. We frame the problem as classification: Buy (1) when the short-term price trend is above the long-term trend, otherwise Sell (0). We also use regression to calculate the expected return. The data file (BitstampData_sample.csv) holds one asset, Bitcoin, with one row per minute from 31 Dec 2011 to 12 Dec 2012 (499,999 rows: Open, High, Low, Close, Volume in BTC and USD).

Objectives: (1) perform EDA; (2) build Buy/Sell and expected-return models in XGBoost, Keras, PyTorch with fastai, PyTorch with TabNet and PyTorch with Lightning; (3) compare the frameworks on simplicity, transparency, efficacy, RMSE, R-squared and MAE. **Our additions:** hourly bars built from sparse minute data, a leakage-safe label design, a time-ordered split, an extra 12-hour-ahead experiment, a blend of XGBoost and a neural network, and SHAP explanations.

2. Methodology

Data preparation. Only 24,521 of 499,999 minutes (4.9%) contain a trade. Trades were combined into hourly bars (first, highest, lowest and last price, summed volume, number of active minutes). Of 8,334 hours, 5,125 had trades; the 3,209 empty hours were dropped instead of inventing prices.

Features (26). All use only current and earlier bars: returns over 1 to 12 bars; price relative to 5, 20 and 30-bar exponential averages; rate of change; RSI, stochastic %K/%D and MACD; volatility and candle shape; volume change, relative volume, active minutes and time since the previous bar; hour of day. Ratios are used instead of raw prices because the price moved from $3.80 to $16.41.

Targets. (a) *Buy/Sell now:* 1 if the 10-bar average price is above the 60-bar average, else 0; these two averages are not used as inputs. (b) *Buy/Sell 12 bars ahead* (extra experiment): the same signal 12 bars later. (c) *Expected return:* next bar's return, capped at plus or minus 5% (no test bar exceeded the cap). After removing the first 60 and last 12 bars, 5,054 rows remain.

Split. Time order, never shuffled, with a 12-bar gap between sets to prevent leakage. Validation was used only for early stopping; the test set was scored once at the end.

Set	Dates (2012)	Rows	Buy share	Return std
Train	11 Jan to 28 Sep	3,525	65.2%	1.52%
Validation	29 Sep to 4 Nov	746	46.0%	1.14%
Test	5 Nov to 11 Dec	759	84.5%	0.83%

Frameworks. All neural networks share one small design (64 and 32 units, dropout 0.3, weight decay, early stopping) and use features standardised with training statistics only.

Framework	Model and settings	Size
XGBoost	Boosted trees, depth 3, rate 0.03, early stopping	56 to 344 trees
Keras	26-64-32-1, ReLU, dropout 0.3, L2, Adam	3,841 weights
PyTorch + fastai	tabular_learner, layers 64 and 32	about 4,000 weights
PyTorch + TabNet	8-wide, 3 decision steps	about 4,300 weights
PyTorch Lightning	Same 64-32 network as a LightningModule	3,841 weights

The brief names Lightning Flash, which is no longer maintained and could not be installed on current Python, so PyTorch Lightning (the library Flash is built on) was used.

Evaluation. Regression: RMSE, MAE, R-squared (baseline: predict the training average, R-squared 0). Classification: accuracy, balanced accuracy, F1, ROC-AUC (baselines: always Buy for the current signal; repeat the current signal for the 12-ahead task). Because the test set is 84.5% Buy, accuracy alone is misleading.

3. Exploratory data analysis

The hourly data has 5,125 rows with no duplicates, missing values or rows where High is below Low. Prices range from $3.80 to $16.41. Hourly volume is very skewed (mean 100.6 BTC, median 35.1 BTC). The market was thin early in 2012: 155 bars with trades in January against 662 in August, and 23.1% of bars follow a gap of two or more hours (longest 29 hours). The average hourly return is 0.039% with a standard deviation of 1.87%. The largest gain (+26.8%) and drop (-22.3%) both occurred on 19 January within three hours; they come from single small trades in a thin market, so they were kept. In total 127 bars moved more than 5%.

The Buy/Sell signal is 65.4% Buy and very persistent: the next bar has the same signal 97.7% of the time (81.9% for 12 bars later), so the honest baseline for forecasting it is repeating the current signal, not 50%. MACD-based features correlate most with the current signal (0.60), which is expected because they use the same prices. For the next-bar return the strongest relationships are negative (-0.36 for price vs its 5-bar average, -0.34 for the latest return): after a sharp move the next bar tends to move back, a typical thin-market effect.

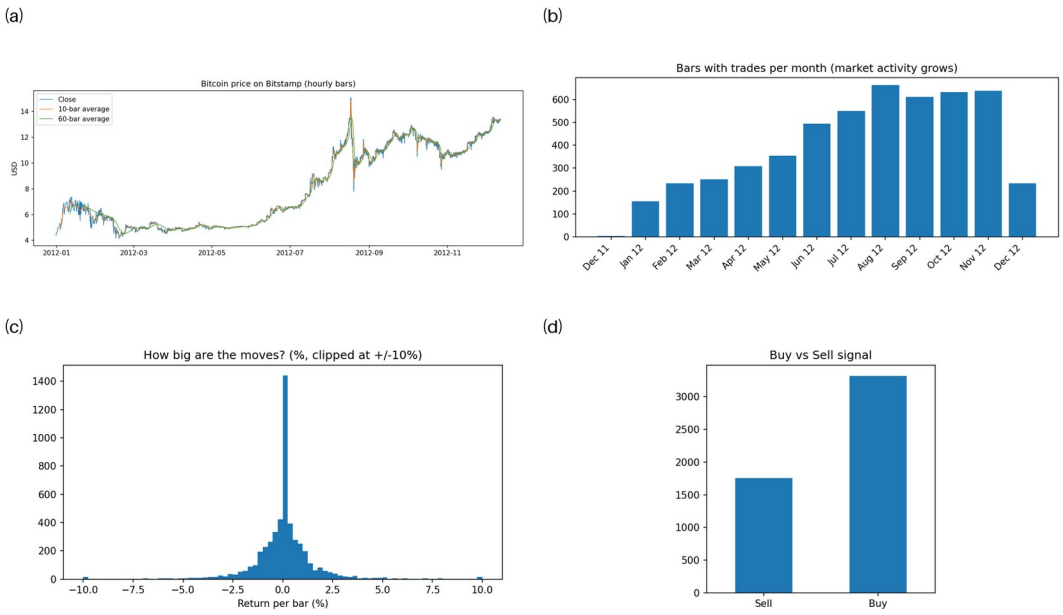


Figure 1: (a) price with 10-bar and 60-bar averages, (b) bars with trades per month, (c) distribution of hourly returns, (d) Buy vs Sell balance

4. Results

All scores are on the held-out test set (5 Nov to 11 Dec 2012, 759 bars), from one final run. “Now” is the current Buy/Sell signal; “12h” is the signal 12 bars ahead.

Model	RMSE	MAE	R^2	Acc. now	F1 now	Acc. 12h	AUC 12h
Baseline	0.0083	0.0056	0.000	84.5%	0.916	84.5%	0.704
XGBoost	0.0073	0.0052	0.217	95.8%	0.975	84.7%	0.825
Keras	0.0074	0.0053	0.210	96.2%	0.977	84.1%	0.825
PyTorch + fastai	0.0073	0.0053	0.216	94.1%	0.965	82.1%	0.825
PyTorch + TabNet	0.0073	0.0053	0.218	96.1%	0.977	85.4%	0.828
PyTorch Lightning	0.0073	0.0052	0.234	96.3%	0.978	83.5%	0.829
Blend (XGB + Keras)	0.0073	0.0052	0.223	96.8%	0.981	85.0%	0.830

Regression. Every framework cuts the error by about 12% and explains about a fifth of the variation in next-hour returns. Differences between frameworks are within noise for 759 bars. Validation R-squared was 0.10 to 0.13; the higher test values reflect a calmer period.

Buy/Sell now. All models beat the always-Buy baseline, also on balanced accuracy (0.89 to 0.93 against 0.50), so the result is not just the uptrend. Because label and features come from the same prices, this is trend recognition, not forecasting.

12 bars ahead. On validation the models beat the 78.9% baseline by about five points, but on test (baseline 84.5%) accuracy ranges from 82.1% to 85.4%, which is not clearly better (a gap of about 1.3 points is noise here), and balanced accuracy is about the same as the baseline. Only AUC stays clearly higher (0.82 to 0.83 against 0.70), so the models rank bars sensibly but do not improve the final decision. The validation gain was not confirmed.

Comparison criteria.

	XGBoost	Keras	fastai	TabNet	Lightning
Training time (s)	0.2 to 0.4	4.8 to 7.5	11 to 17	10 to 14	3.5 to 6.7
Simplicity	Few settings, no scaling	Simple API, needs scaling	Shortest set-up, hidden defaults	Most settings	Most boilerplate
Transparency	High: importance and SHAP	Low	Low	Medium: feature masks	Low
Efficacy	Within noise	Within noise	Lowest accuracy now and 12h	Within noise	Best R^2, within noise

Blending and SHAP. Averaging XGBoost and Keras was level with the best single model (validation R-squared 0.116 against 0.103 and 0.115; test 0.223), so blending gives little reliable gain. SHAP on XGBoost shows that when the latest return or the price distance above its 5-bar average is high, the predicted next return is lower, which matches the bounce-back seen in the EDA (Figure 2). For the current-signal model the top features are MACD variants, i.e. trend recognition. XGBoost's built-in importance and SHAP agree broadly (rank correlation 0.84, 0.67, 0.84 for the three tasks).

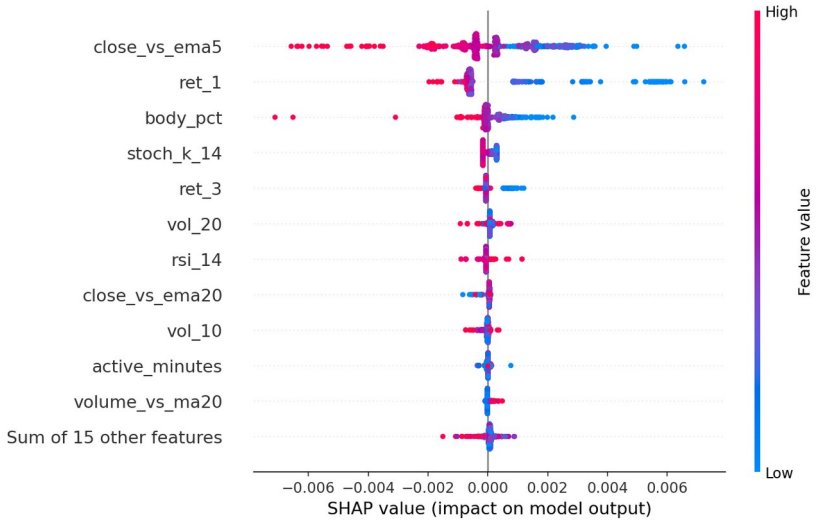


Figure 2: SHAP summary plot for the regression model

5. Conclusion

1. All five frameworks recognise the current Buy/Sell trend accurately (94 to 97% against 84.5%).
2. Next-hour return is only slightly predictable (R-squared 0.21 to 0.23, error about 12% lower), mostly through a thin-market bounce-back that is not a trading strategy.
3. The 12-hour-ahead forecast did not clearly beat repeating the current signal on test.
4. The frameworks perform almost identically; XGBoost is the fastest (under 0.5 s against 4 to 17 s) and the easiest to explain, so it is the best choice for this size of tabular data. Blending gave no reliable improvement.

Limitations. One year of a thin early market; empty hours dropped, so bars are unevenly spaced; a single split whose three periods behave very differently; label and features share the same prices; no trading costs or back-test, so no profitability claim; small fixed hyperparameters; Lightning used in place of Flash. **Future work:** rolling validation, later data, a fee-aware back-test, hyperparameter search and a three-class signal (Buy, Sell, no position).

References. Course project brief (Project Id 3). Bitstamp sample data: github.com/tiwvis/MLBD-Projects. Libraries: XGBoost, Keras/TensorFlow, fastai, pytorch-tabnet, PyTorch Lightning, SHAP, scikit-learn.